JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class FutureTask<V>

java.lang.Object
    java.util.concurrent.FutureTask<V>

Type Parameters:

`V - The result type returned by this FutureTask's get methods`

All Implemented Interfaces:

`Runnable, Future<V>, RunnableFuture<V>`

```
public class FutureTask<V>
extends Object
implements RunnableFuture<V>
```

A cancellable asynchronous computation. This class provides a base implementation of `Future`, with methods to start and cancel a computation, query to see if the computation is complete, and retrieve the result of the computation. The result can only be retrieved when the computation has completed; the `get` methods will block if the computation has not yet completed. Once the computation has completed, the computation cannot be restarted or cancelled (unless the computation is invoked using `runAndReset()`).

A `FutureTask` can be used to wrap a `Callable` or `Runnable` object. Because `FutureTask` implements `Runnable`, a `FutureTask` can be submitted to an `Executor` for execution.

In addition to serving as a standalone class, this class provides `protected` functionality that may be useful when creating customized task classes.

Since:

`1.5`

Constructor Summary

Constructors

Constructor	Description
`FutureTask(Callable<V> callable)`	Creates a `FutureTask` that will, upon running, execute the given `Callable`.
`FutureTask(Runnable runnable, V result)`	Creates a `FutureTask` that will, upon running, execute the given `Runnable`, and arrange that `get` will return the given result on successful completion.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`boolean`	`cancel(boolean mayInterruptIfRunning)`	Attempts to cancel execution of this task.
`protected void`	`done()`	Protected method invoked when this task transitions to state `isDone` (whether normally or via cancellation).
`V`	`get()`	Waits if necessary for the computation to complete, and then retrieves its result.
`V`	`get(long timeout, TimeUnit unit)`	Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.

boolean	**isCancelled**()	Returns `true` if this task was cancelled before it completed normally.
boolean	**isDone**()	Returns `true` if this task completed.
void	**run**()	Sets this Future to the result of its computation unless it has been cancelled.
protected boolean	**runAndReset**()	Executes the computation without setting its result, and then resets this future to initial state, failing to do so if the computation encounters an exception or is cancelled.
protected void	**set**(**V** v)	Sets the result of this future to the given value unless this future has already been set or has been cancelled.
protected void	**setException**(**Throwable** t)	Causes this future to report an **ExecutionException** with the given throwable as its cause, unless this future has already been set or has been cancelled.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

FutureTask

```
public FutureTask(Callable<V> callable)
```

Creates a `FutureTask` that will, upon running, execute the given `Callable`.

Parameters:

`callable - the callable task`

Throws:

`NullPointerException - if the callable is null`

FutureTask

```
public FutureTask(Runnable runnable,
                  V result)
```

Creates a `FutureTask` that will, upon running, execute the given `Runnable`, and arrange that `get` will return the given result on successful completion.

Parameters:

`runnable - the runnable task`

`result - the result to return on successful completion. If you don't need a particular result, consider using constructions of the form: Future<?> f = new FutureTask<Void>(runnable, null)`

Throws:

`NullPointerException - if the runnable is null`

Method Detail

isCancelled

```
public boolean isCancelled()
```

Description copied from interface: `Future`

Returns `true` if this task was cancelled before it completed normally.

Specified by:

`isCancelled in interface Future<V>`

Returns:

`true if this task was cancelled before it completed`

isDone

```
public boolean isDone()
```

Description copied from interface: `Future`

Returns `true` if this task completed. Completion may be due to normal termination, an exception, or cancellation -- in all of these cases, this method will return `true`.

Specified by:

`isDone` in interface `Future<V>`

Returns:

`true if this task completed`

cancel

```
public boolean cancel(boolean mayInterruptIfRunning)
```

Description copied from interface: `Future`

Attempts to cancel execution of this task. This attempt will fail if the task has already completed, has already been cancelled, or could not be cancelled for some other reason. If successful, and this task has not started when `cancel` is called, this task should never run. If the task has already started, then the `mayInterruptIfRunning` parameter determines whether the thread executing this task should be interrupted in an attempt to stop the task.

After this method returns, subsequent calls to `Future.isDone()` will always return `true`. Subsequent calls to `Future.isCancelled()` will always return `true` if this method returned `true`.

Specified by:

`cancel` in interface `Future<V>`

Parameters:

`mayInterruptIfRunning - true if the thread executing this task should be interrupted; otherwise, in-progress tasks are allowed to complete`

Returns:

`false if the task could not be cancelled, typically because it has already completed normally; true otherwise`

get

```
public V get()
      throws InterruptedException,
             ExecutionException
```

Description copied from interface: Future

Waits if necessary for the computation to complete, and then retrieves its result.

Specified by:

`get in interface Future<V>`

Returns:

`the computed result`

Throws:

`CancellationException - if the computation was cancelled`

`InterruptedException - if the current thread was interrupted while waiting`

`ExecutionException - if the computation threw an exception`

get

```
public V get(long timeout,
             TimeUnit unit)
      throws InterruptedException,
             ExecutionException,
             TimeoutException
```

Description copied from interface: Future

Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.

Specified by:

`get in interface Future<V>`

Parameters:

`timeout - the maximum time to wait`

`unit - the time unit of the timeout argument`

Returns:

`the computed result`

Throws:

`CancellationException - if the computation was cancelled`

`InterruptedException - if the current thread was interrupted while waiting`

`ExecutionException` - if the computation threw an exception

`TimeoutException` - if the wait timed out

done

```
protected void done()
```

Protected method invoked when this task transitions to state `isDone` (whether normally or via cancellation). The default implementation does nothing. Subclasses may override this method to invoke completion callbacks or perform bookkeeping. Note that you can query status inside the implementation of this method to determine whether this task has been cancelled.

set

```
protected void set(V v)
```

Sets the result of this future to the given value unless this future has already been set or has been cancelled.

This method is invoked internally by the `run()` method upon successful completion of the computation.

Parameters:

`v` - the value

setException

```
protected void setException(Throwable t)
```

Causes this future to report an `ExecutionException` with the given throwable as its cause, unless this future has already been set or has been cancelled.

This method is invoked internally by the `run()` method upon failure of the computation.

Parameters:

`t` - the cause of failure

run

```
public void run()
```

Description copied from interface: RunnableFuture

Sets this Future to the result of its computation unless it has been cancelled.

Specified by:

`run` in interface `Runnable`

Specified by:

`run` in interface `RunnableFuture<V>`

See Also:

`Thread.run()`

runAndReset

```
protected boolean runAndReset()
```

Executes the computation without setting its result, and then resets this future to initial state, failing to do so if the computation encounters an exception or is cancelled. This is designed for use with tasks that intrinsically execute more than once.

Returns:

`true if successfully run and reset`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.